

# Dropout1d#

<https://pytorch.org/docs/stable/generated/torch.nn.Dropout1d.html>

## Description:

Randomly zero out entire channels. A channel is a 1D feature map, e.g., the  $j$ -th channel of the  $i$ -th sample in the batched input is a 1D tensor  $\text{input}[i, j]$ . Each channel will be zeroed out independently on every forward call with probability  $p$  using samples from a Bernoulli distribution. Usually the input comes from `nn.Conv1d` modules. As described in the paper *Efficient Object Localization Using Convolutional Networks*, if adjacent pixels within feature maps are strongly correlated (as is normally the case in early convolution layers) then i.i.d. dropout will not regularize the activations and will otherwise just result in an effective learning rate decrease.

## Function Signature

---

```
class torch.nn.Dropout1d(p=0.5, inplace=False)[source]#
```

Parameters

Shape:

```
forward(input)[source]#
```

Return type

**Returns:**

Tensor

## Code Examples

---

```
>>> m = nn.Dropout1d(p=0.2)
>>> input = torch.randn(20, 16, 32)
>>> output = m(input)
```

# Detailed Documentation

---

## Documentation

Randomly zero out entire channels.

A channel is a 1D feature map, e.g., the  $j$ -th channel of the  $i$ -th sample in the batched input is a 1D tensor  $\text{input}[i, j]$ .

Each channel will be zeroed out independently on every forward call with probability  $p$  using samples from a Bernoulli distribution.

Usually the input comes from `nn.Conv1d` modules.

As described in the paper *Efficient Object Localization Using Convolutional Networks*, if adjacent pixels within feature maps are strongly correlated (as is normally the case in early convolution layers) then i.i.d. dropout will not regularize the activations and will otherwise just result in an effective learning rate decrease.

In this case, `nn.Dropout1d()` will help promote independence between feature maps and should be used instead.

$p$  (float, optional) – probability of an element to be zero-ed.

`inplace` (bool, optional) – If set to `True`, will do this operation in-place

Input:  $(N, C, L)$  or  $(C, L)$

Output:  $(N, C, L)$  or  $(C, L)$  (same shape as input).

Runs the forward pass.

